0/1 KNAPSACK USING DYNAMIC PROGRAMMING

Theory • Algorithm • Example • Complexity • Viva

1. Theory

The **0/1 Knapsack Problem** is an optimization problem in which each item has a profit and a weight. The objective is to select items so that the total profit is maximum while the total weight does not exceed the knapsack's capacity. In this problem, each item has only two choices: it can either be selected completely (1) or not selected at all (0). Items cannot be divided into fractions.

The problem can be solved using **Dynamic Programming**. A table is created to store the maximum profit achievable using the first i items for each possible capacity. For every item, the algorithm compares two choices: including the item or excluding it. The larger profit is stored in the table. This avoids recalculating the same subproblems.

2. Dynamic Programming Recurrence

Let **DP[i][w]** represent the maximum profit obtainable using the first i items with knapsack capacity w.

If the current item's weight is greater than the available capacity:
DP[i][w] = DP[i-1][w]

Otherwise, choose the better of including or excluding the item:
DP[i][w] = max(profit[i-1] + DP[i-1][w-weight[i-1]], DP[i-1][w]).

3. Algorithm Steps

1. Accept the number of items, their profits, and their weights.

2. Accept the knapsack capacity.

3. Create a DP table of size (n + 1) × (capacity + 1), initialized with zeros.

4. Process each item and each possible capacity.

5. If the item's weight exceeds the current capacity, copy the value from the previous row.

6. Otherwise, compare the profit from including the item with the profit from excluding it.

7. Store the larger value in the DP table.

8. The value in DP[n][capacity] is the maximum profit.

4. Example

Suppose there are 3 items and the knapsack capacity is 50.

Item	Profit	Weight
1	60	10
2	100	20
3	120	30

The optimal selection is Item 2 and Item 3.
Total weight = 20 + 30 = 50
Total profit = 100 + 120 = **220**.

5. Complexity Analysis

Type	Complexity	Explanation
Time	O(n × W)	The DP table has n × W states, and each state is computed in constant time.
Space	O(n × W)	The algorithm stores a table with (n + 1) rows and (W + 1) columns.

6. Applications

• Budget allocation and resource planning.
• Cargo loading and capacity planning.
• Selecting projects within a limited budget.
• Choosing items under storage or weight constraints.

7. Main Theory to Remember

0/1 Knapsack is solved using Dynamic Programming. Each item is either selected completely or rejected; fractions are not allowed. The algorithm builds a DP table and compares including an item with excluding it to obtain the maximum profit without exceeding capacity.

8. Viva Questions

Q1. Why is it called 0/1 Knapsack?
Because each item is either selected (1) or not selected (0).

Q2. Which technique is used?
Dynamic Programming.

Q3. Can an item be divided?
No, only complete items can be selected.

Q4. What does DP[i][w] represent?
The maximum profit using the first i items with capacity w.

Q5. What is the time complexity?
O(n × W).

EXAM-READY LINE

The 0/1 Knapsack problem uses Dynamic Programming to find the maximum profit by either including or excluding each complete item without exceeding the given capacity.